A handful of organizations have deployed reinforcement learning beyond simulation, achieving measurable gains on specific large-scale problems. Each deployment required substantial domain engineering and scientific tuning; these remain exceptional cases rather than standard practice. I review the most prominent field deployments, including ride-hailing dispatch at DiDi, data center cooling at Google, hotel revenue management, and financial order execution, before concluding with a simulation study on the bus engine replacement problem. In each case, the RL agent's parameters were updated during a training phase conducted in simulation or on historical data, and the resulting policy was deployed with fixed weights (Section **??**), with the exception of the hotel revenue management system, which learned in-field.

0.1 Ride-Hailing Dispatch

Each driver-passenger assignment changes the spatial distribution of available drivers, making ride-hailing dispatch a sequential optimization problem at a scale (tens of millions of daily rides at DiDi) where exact dynamic programming is intractable.

Qin et al. (2021) formalized DiDi's order dispatching as a semi-Markov decision process[1] where each driver is an independent agent. The state of a driver consists of location (discretized into hexagonal zones) and time (bucketed into intervals). The action is the order assigned to the driver, with the option to remain idle. The reward is the trip fare. State transitions are determined by trip destinations, as completing an order transports the driver from origin to destination, changing their spatial state. The stochasticity arises from future demand, which determines the available actions at each state. The per-driver value function $V^\pi(s)$ represents the expected cumulative fare a driver can earn from state s under dispatching policy π:

$$V^\pi(s) = \mathbb{E}\left[\sum_{k=0}^{\infty} \gamma^{\tau_k} r_k \mid s_0 = s, \pi\right], \tag{1}$$

where τ_k is the time to complete the k-th trip and r_k is the corresponding fare. The platform's objective is to maximize total driver income across the fleet, which decomposes into the sum of individual driver value functions.

Each dispatching window (a few seconds), the platform collects open orders and avail-

[1] A semi-Markov decision process generalizes the standard MDP by allowing variable time between decisions. The discount factor γ^{τ_k} depends on the actual time elapsed τ_k rather than applying a fixed per-step discount.

able drivers, constructs a bipartite graph, and solves a linear assignment problem. The edge weights are computed as the advantage of each driver-order match relative to the driver's current state value.

$$w_{ij} = \hat{r}_i + \gamma^{\hat{\tau}_i} V(s'_j) - V(s_j), \tag{2}$$

where $\hat{r}_i$ is the predicted fare for order i, $\hat{\tau}_i$ is the estimated trip duration, and s'_j is the destination state. The value function is learned via temporal-difference methods from historical trip data. DiDi's Cerebellar Value Network (CVNet; Tang et al., 2019) uses hierarchical coarse-coding[2] with a multi-resolution hexagonal grid, enabling transfer learning[3] across cities and robustness to data sparsity in low-traffic zones.

Production deployment across more than 20 Chinese cities demonstrated modest but consistent improvements of 0.5–2% on key metrics, gains that required the full CVNet infrastructure (hierarchical coarse-coding, multi-resolution grids, transfer learning across cities) to achieve. Table 1 summarizes the reported gains from A/B tests using time-slice rotation, where algorithms alternate control of the platform in 3-hour blocks to avoid interference effects.

Table 1: DiDi dispatch deployment results from Qin et al. (2021) and Tang et al. (2019).

Metric	Improvement vs. Baseline	Scale
Total driver income	+0.5–2.0%	Tens of millions daily rides
Order response rate	+0.5–2.0%	Platform-wide
Fulfillment rate	+0.5–2.0%	Platform-wide

Li et al. (2019) addressed the coordination challenge using mean-field multi-agent RL. With thousands of drivers making simultaneous decisions, the full multi-agent state space is intractable. The mean-field approximation replaces individual driver states with an aggregate distribution, allowing each driver to condition on the density of nearby drivers rather than their exact locations. Experiments on DiDi data showed improved fleet utilization compared to single-agent baselines.

Han et al. (2022) reported complementary results from Lyft, where the dispatching

[2] *Coarse-coding* represents a value function as a weighted sum over overlapping rectangular or hexagonal tiles that partition the state space (Sutton and Barto, 2018); each state activates the tiles that contain it, and a hierarchical variant stacks tiles at multiple resolutions to allow both coarse and fine generalization simultaneously.

[3] *Transfer learning* initializes a model for a new task (a new city) with parameters trained on related tasks (existing cities), on the assumption that learned representations of traffic and demand patterns generalize across contexts and require less data to reach good performance in the new setting.

system optimizes driver assignment and repositioning jointly. Their value decomposition architecture[4] assigns credit to individual driver decisions within the global objective. The production system demonstrated improvements in rider wait times and driver utilization, providing a second data point suggesting that similar approaches may transfer across platforms.

At DiDi's volume, even 1% gains represent hundreds of thousands of additional completed rides per day, because dispatching is fundamentally a fleet positioning problem: today's assignments determine tomorrow's driver distribution.

0.2 Hotel Revenue Management

Budget hotel chains face a capacity allocation problem: how to dynamically distribute rooms across rate segments defined by discount level, with booking channels such as direct platforms and online travel agencies mapped to segments offering discounts ranging from less than 15% to over 40%. Demand is uncertain, cancellations are difficult to forecast, and hotel managers resist black-box optimization systems that override their judgment. Chen et al. (2023) deployed reinforcement learning at China Lodging Group (CLG), a budget hotel chain operating approximately 2,000 hotels across China, and conducted field experiments measuring the impact of RL-based capacity allocation on actual hotel revenue.

Their system uses a two-step design. The RL agent observes the state (t, s), where t indexes the booking period within a $T = 10$-period episode and s is the average revenue per room sold to date, and selects an average discount level $a \in \{10\%, 20\%, 30\%\}$. A linear program then converts this scalar recommendation into a feasible capacity allocation across rate segments, accounting for each hotel's channel preferences.[5] This decomposition addresses practitioner acceptance, since managers understand a single discount recommendation, and circumvents the need to explicitly model demand arrivals or cancellation rates.

The field experiment randomly assigned five Hanting-brand hotels in Shanghai to the

[4]Value decomposition decomposes the platform's global matching objective into individual driver value functions, each estimable via temporal-difference learning. The global optimum is recovered by optimizing the sum of individual values.

[5]The RL component uses a modified on-policy Monte Carlo method with ε-greedy exploration, updating Q-values from realized returns after each completed episode, making it an in-field learner in the terminology of Section **??**.

treatment group from ten candidates, with 271 additional Shanghai hotels serving as a donor pool for synthetic control estimation.[6] Table 2 reports the average treatment effects over the pilot period (March–June 2015).

Table 2: Field experiment results from Chen et al. (2023), measured via synthetic control.

Metric	Improvement	p-value
Occupancy rate (OR)	+5.15%	0.1361
Average daily rate (ADR)	+5.93%	0.1160
Revenue per available room (RevPAR)	+11.80%	0.0090

The RevPAR gain is heterogeneous across treatment hotels; some improved primarily via higher occupancy rates, others via higher average daily rates, and others via both channels simultaneously.

0.3 E-Commerce Dynamic Pricing

E-commerce platforms face a pricing problem at a scale that defeats human specialists. Alibaba's Tmall.com lists millions of SKUs across thousands of product categories, each requiring daily price adjustments that account for demand elasticity, competitor behavior, inventory levels, and promotional calendars. Liu et al. (2019) deployed deep reinforcement learning agents for automated pricing on Tmall.com beginning July 2018, conducting field experiments on thousands of SKUs over several months.

The pricing MDP for product i has state $s_{i,t} \in \mathbb{R}^m$ comprising four feature groups: price features (current and historical prices, price-to-cost ratio), sales features (units sold, conversion rate), customer traffic features (unique visitors $\text{uv}_{i,t}$, page views), and competitiveness features (price rank among similar products). The pricing period is $d = 1$ day. Actions are either discrete, $a_{i,t} \in \{1, \ldots, K\}$ indexing price bins uniformly spaced between product-specific bounds $[P_{i,\min}, P_{i,\max}]$, or continuous, $a_{i,t} \in \mathbb{R}$. The reward is the difference of revenue conversion rates (DRCR):

$$r_{i,t} = \frac{\text{revenue}_{i,t}}{\text{uv}_{i,t}} - \frac{\text{revenue}_{i,t-\tau}}{\text{uv}_{i,t-\tau}}, \tag{3}$$

where $\text{uv}_{i,t}$ is unique visitors in period t and τ is a reference lag.[7]

[6] Synthetic control constructs a weighted combination of untreated hotels whose pre-treatment trend matches each treated hotel, avoiding the selection bias of simple before-after comparisons.

[7] DRCR normalizes revenue by traffic to remove demand fluctuations unrelated to pricing. The differencing

Two algorithms are deployed: DQN for the discrete action formulation and DDPG for continuous actions. Both are pre-trained from demonstrations using historical specialist pricing actions (DQfD, DDPGfD) to address cold-start.[8][9]

Table 3: Field experiment results from Liu et al. (2019) on Tmall.com. DRCR improvement is relative to the simi-product control group.

Experiment	Method	DRCR Improvement	Duration
Markdown (500 luxury SKUs)	DQN ($K = 9$)	+37.5%	15 days
Daily (1000 FMCGs)	DQN ($K = 100$)	5.10×	30 days
Daily (1000 FMCGs)	DDPG (continuous)	6.07×	30 days

DDPG with continuous action space outperformed DQN with discrete bins across all daily pricing experiments, and both substantially outperformed manual expert pricing.

0.4 Financial Order Execution

The theoretical benchmark for optimal execution is the Almgren-Chriss framework (Almgren and Chriss, 2001), which derives optimal deterministic schedules under linear impact assumptions. A trader liquidating Q shares over T periods faces a tradeoff between timing risk and market impact. With risk-aversion parameter λ, price volatility σ, and temporary impact coefficient η, the optimal remaining inventory at time t follows the hyperbolic sine schedule:

$$x^*(t) = Q \cdot \frac{\sinh\big(\kappa(T-t)\big)}{\sinh(\kappa T)}, \quad \kappa = \sqrt{\frac{\lambda\sigma^2}{\eta}}. \tag{4}$$

This deterministic trajectory is the benchmark any adaptive method must beat. It prescribes front-loading or back-loading depending on the risk-impact balance, but cannot respond to realized order flow or spread dynamics.

Nevmyvaka et al. (2006) applied tabular Q-learning to execution on real limit order book data from NASDAQ stocks.[10] The state at time t is $s_t = (t, q_t, \psi_t, \Delta_t)$, where

further removes product-specific level effects, yielding a reward signal that is more concave than raw revenue and improves convergence stability.

[8] Pre-populating replay buffers with rollouts from heuristic or scripted policies is a general technique for reducing exploration burden in deep RL. In robotic grasping, Kalashnikov et al. (2018) collected 200,000 scripted grasping attempts at 15–30% success rate, sufficient to bootstrap a vision-based policy to 96% success; see Ibarz et al. (2021) for a survey of these and related warm-start strategies.

[9] Standard A/B testing is infeasible because Chinese e-commerce regulations prohibit displaying different prices to different customers for the same product simultaneously. Liu et al. (2019) instead evaluate using difference-in-differences against "simi-products" (similar products not subject to algorithmic pricing) as controls.

[10] The dataset covers 500 trading days of millisecond-level limit order book snapshots for AMZN, QCOM, and NVDA. The state space contains approximately 10,000 states; the horizon is $T = 60$ seconds with discount $\gamma = 1$.

$t \in \{1, \ldots, T\}$ is time remaining, $q_t \in \{0, 1, \ldots, Q\}$ is inventory remaining, ψ_t is the discretized bid-ask spread, and Δ_t is the discretized signed volume imbalance.[11] Actions $a_t \in \{0, \delta, 2\delta, \ldots, q_t\}$ specify shares to execute in the current interval. The reward is the negative per-period slippage contribution:

$$r_t = -a_t\Big(p_t^{\text{exec}} - p_0\Big), \tag{5}$$

where p_0 is the mid-quote at order arrival and p_t^{exec} is the average execution price. Total implementation shortfall is $IS = -\sum_{t=1}^{T} r_t/Q$.[12] Because the objective is cost minimization, the Q-learning update uses min over next-period actions:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\Big[r_t + \gamma \min_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)\Big]. \tag{6}$$

The agent learns to condition on market microstructure signals: trading aggressively when spreads are narrow, waiting when order flow predicts favorable price movement, and accelerating near the deadline.

Table 4: Execution results from Nevmyvaka et al. (2006) on NASDAQ stocks. TWAP is the time-weighted average price baseline (equal-sized trades at uniform intervals).

Stock	RL vs. TWAP	RL vs. Almgren-Chriss	Data
AMZN	−18.2%	−14.6%	500 days LOB
QCOM	−22.1%	−19.3%	500 days LOB
NVDA	−15.8%	−12.1%	500 days LOB

The RL agent reduces execution costs by 12–19% over Almgren-Chriss. These results informed subsequent work on adaptive execution, though independently verified production deployments remain scarce in the public literature.

0.5 Supply Chain Inventory Management

Multi-echelon inventory systems coordinate ordering decisions across supply chain stages, where each stage's order becomes the next stage's incoming shipment. A retailer orders from a warehouse, which orders from a distribution center, which orders from a factory.

[11] Signed volume imbalance is the difference between buy and sell order volume near the best prices, normalized by total volume; positive imbalance typically predicts short-term price increases.

[12] Implementation shortfall measures the total cost of executing a trade relative to the benchmark mid-quote at arrival. It captures market impact, timing cost, and opportunity cost of unexecuted shares.

The sequential nature creates complex dependencies, since an order placed upstream today affects downstream availability many periods later. The state space grows exponentially in the number of echelons, with K stages each having M possible inventory levels yielding M^K states. Classical inventory theory provides closed-form solutions for special cases, most notably the echelon base-stock policy of Clark and Scarf (1960).

The state $s = (I_1, \ldots, I_K)$ records on-hand inventory at each stage, where stage 1 faces customer demand and stage K is the most upstream. The action $q \in \{0, 1, \ldots, Q_{\max}\}$ is the order quantity placed at stage K. Demand $D_t \sim F_D$ arrives at stage 1 each period; unfilled demand is backordered. Shipments flow downstream, with stage k receiving what stage $k+1$ shipped in the previous period. The per-period cost combines holding, backorder, and ordering components.

$$c_t = h \sum_{k=1}^{K} I_k^+ + b \cdot (D_t - I_1)^+ + c_o \cdot q_t, \tag{7}$$

where $I_k^+ = \max(0, I_k)$ is on-hand inventory, $(x)^+ = \max(0, x)$, h is holding cost per unit, b is backorder cost per unit, and c_o is ordering cost. The objective is to minimize expected discounted total cost.

Gijsbrechts et al. (2022) conducted a systematic evaluation of deep RL against classical base-stock policies across lost sales, dual sourcing, and multi-echelon configurations. The classical benchmark is the echelon base-stock policy, in which each stage k maintains an echelon inventory position and orders to bring this position to a target level S_k.[13] For single-echelon systems with backorders, the optimal base-stock level is given by the newsvendor critical fractile $S^* = F_D^{-1}(b/(b+h))$. Table 5 summarizes results from their multi-echelon experiments.

Table 5: Multi-echelon inventory results from Gijsbrechts et al. (2022).

Echelons	DRL Cost Gap vs. Base-Stock	States	Convergence
2	+1.2%	$\sim 10^2$	Achieved
3	+6.8%	$\sim 10^3$	Achieved
4	+12.3%	$\sim 10^5$	Partial
6	—	$\sim 10^8$	Failed

[13] Formally, the echelon inventory position at stage k equals on-hand inventory at k plus all inventory at downstream stages $1, \ldots, k-1$ plus in-transit inventory, minus backorders. The echelon base-stock policy of Clark and Scarf (1960) is optimal for serial systems with linear costs and backorders.

Where classical solutions exist, RL underperforms: the base-stock policy remains highly competitive when properly calibrated, and DRL required millions of training transitions to approach performance levels achievable through closed-form calculation. RL struggles particularly with multi-echelon coupling, where upstream orders affect downstream costs many periods later.[14] The value proposition for RL lies in problems where analytical solutions are unavailable: non-stationary demand, complex operational constraints, or cost structures that do not admit tractable decomposition.[15] Even in these settings, successful deployment remains rare and requires extensive simulation infrastructure, domain expertise, and careful calibration against classical baselines.

0.6 Real-Time Bidding

Real-time bidding for display advertising presents a budget pacing problem: an advertiser must allocate a fixed budget B_0 across a campaign of T auctions to maximize total conversions. Each impression is a second-price auction; the advertiser submits a bid and pays the second-highest bid if they win. The challenge is that bidding aggressively depletes budget early, while conservative bidding leaves value on the table.

Wu et al. (2018) formalized this as an MDP with state $s_t = (B_t, t, w_t)$, where B_t is remaining budget, t is auctions remaining, and w_t is the recent win rate. The action is a bid multiplier $\lambda_t \in [\lambda_{\min}, \lambda_{\max}]$, so the actual bid is $b_t = \lambda_t \cdot \bar{b}$, where $\bar{b}$ is a base bid calibrated to the estimated value-per-impression. The clearing price c_t is drawn from a distribution F_c estimated from historical data; the advertiser wins if $b_t \geq c_t$. The per-auction reward is:

$$r_t = \mathbb{1}\{b_t \geq c_t\} \cdot \text{cvr}_t, \tag{8}$$

where $\text{cvr}_t \in \{0, 1\}$ is the conversion indicator. Budget evolves as $B_{t+1} = B_t - \mathbb{1}\{b_t \geq c_t\} \cdot c_t$, and the episode terminates when $B_t \leq 0$ or $t = 0$.[16]

[14] Credit assignment refers to the difficulty of determining which past actions caused a delayed reward or cost. In multi-echelon systems, an upstream ordering decision today may not affect customer-facing costs for many periods, making it difficult for RL to learn the causal connection.

[15] Residual RL (Silver et al., 2018) offers a middle ground: rather than replacing classical policies, learn a correction $\pi_{\text{final}}(s) = \pi_{\text{classical}}(s) + \pi_{\text{learned}}(s)$, preserving the classical solution's performance while allowing RL to handle constraints or non-stationarity the analytical method cannot. This connects to perturbation methods in applied mathematics, where one linearizes around a known solution and solves for deviations. See Ibarz et al. (2021) for a survey of residual and demonstration-augmented RL in robotics.

[16] The state space is discretized into budget bins and time bins. Wu et al. (2018) augment the state with bid landscape features derived from historical clearing price distributions, giving the agent information about current market competitiveness.

The agent is a deep Q-network trained on a simulator calibrated to production RTB data. Table 6 reports performance relative to rule-based pacing baselines. The DQN agent improves total conversions by conserving budget during high-competition periods and bidding aggressively when clearing prices are low, a pattern the rule-based approaches cannot learn.

Table 6: Real-time bidding results from Wu et al. (2018). Performance is relative to linear pacing on a simulator calibrated to production data.

Method	Conversions vs. Linear Pacing	Budget Utilization
Linear pacing	baseline	98%
Hard threshold	−8.3%	91%
Proportional (ECPC)	+4.1%	97%
DQN (Wu et al., 2018)	+11.7%	99%

0.7 Simulation Study: Bus Engine Replacement

I conclude with a simulation demonstrating that RL matches dynamic programming on a classical benchmark. The bus engine replacement problem, introduced by Rust (1987), models a fleet manager's monthly decision whether to replace engines based on accumulated mileage. Replacement incurs a fixed cost but resets mileage to zero; continued operation incurs maintenance costs increasing in mileage.

I extend the single-engine problem to a fleet of N engines with a capacity constraint limiting replacements per period. The state $s = (m_1, \ldots, m_N)$ records discretized mileage for each engine. Actions are subsets of engines to replace, subject to the capacity constraint. The per-period cost is $c(s, a) = \alpha \sum_i m_i + \beta|a|$, where α is the operating cost per unit mileage and β is the replacement cost.[17] Mileage evolves deterministically; replaced engines reset to $m = 0$; others increment by one bin.[18] I use $\gamma = 0.95$ rather than Rust's monthly $\beta = 0.9999$, which softens the effective discount horizon from roughly 10,000 periods to roughly 20 and stabilises DQN training; the qualitative DQN-vs-DP comparison is unaffected.[19]

Figure 1 compares dynamic programming, DQN, and heuristic baselines across fleet

[17]The mileage-dependent operating cost $\alpha \sum_i m_i$ follows Rust's original specification $c(x, \theta_1) = \theta_{11} x$, creating a non-trivial threshold replacement policy. The fleet extension uses deterministic mileage increments to isolate the combinatorial scaling challenge that arises from the joint state of multiple engines.

[18]With $M = 6$ mileage bins, the state space is 6^N: 1,296 states at $N = 4$, 7,776 at $N = 5$, 46,656 at $N = 6$.

[19]Rust's $\beta = 0.9999$ produces Q-values of order 10^4, which destabilise gradient updates on a small replay buffer. The substantive scaling story (combinatorial blow-up of $|\mathcal{S}|$ with N) is invariant to the discount choice.

sizes. All policies are evaluated on the same pre-sampled set of initial states so that the comparison is paired across methods.

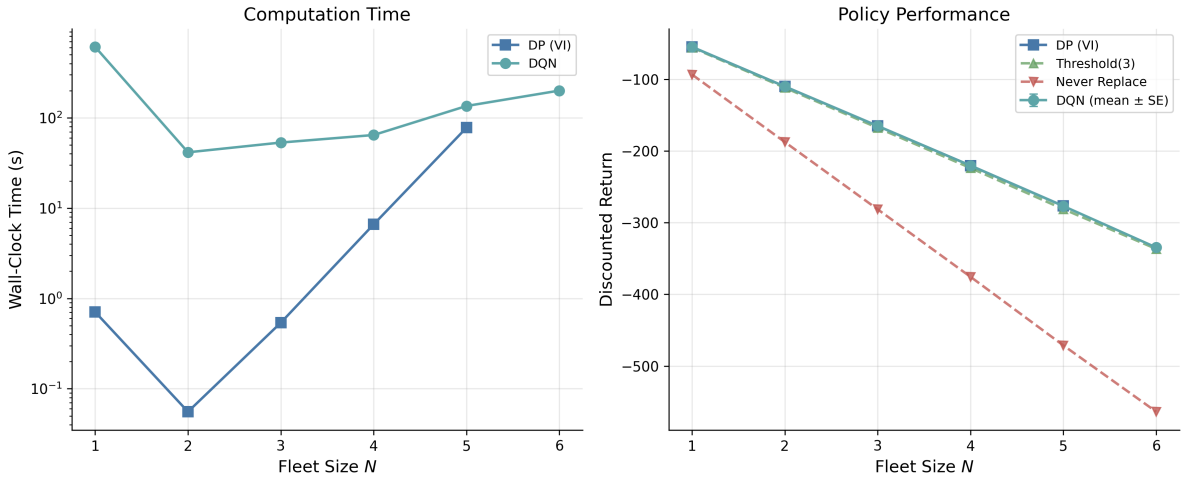

Figure 1: Bus engine replacement benchmark. Left: computation time vs. fleet size (log scale). Right: discounted return vs. fleet size for DP, DQN, and heuristic baselines; DQN error bars are mean ± standard error over ten seeds. At $N = 6$ (46,656 states), DP is omitted because plain-Python value iteration becomes wall-clock prohibitive at this scale.

For $N = 1$ through 5 where both methods are computed, DQN matches DP within 1% of the optimal discounted return. At $N = 6$ (46,656 states), I omit DP rather than run it: plain-Python value iteration is not numerically infeasible at this size, but a single backup sweep costs $|\mathcal{S}|^2|\mathcal{A}|$ in Python-loop time and exceeds the wall-clock budget by an order of magnitude.[20] The threshold heuristic, which replaces engines above a mileage cutoff, provides a reasonable baseline but cannot account for capacity constraints or the joint state of multiple engines. The never-replace heuristic performs poorly due to accumulated mileage costs, confirming that the cost structure creates a non-trivial replacement decision.

[20]The DP wall-clock curve in the left panel reflects this Python-loop implementation; a vectorised backup using `numpy` broadcasts would shrink the absolute timings by one to two orders of magnitude. The scaling exponent (slope of the log-time curve in N) is the right asymptotic shape, but the absolute timings are implementation-dependent.

References

Robert Almgren and Neil Chriss. Optimal execution of portfolio transactions. *Journal of Risk*, 3(2):5–40, 2001.

Ji Chen, Yifan Xu, Peiwen Yu, and Jun Zhang. A reinforcement learning approach for hotel revenue management with evidence from field experiments. *Journal of Operations Management*, 69(7):1176–1201, 2023. doi: 10.1002/joom.1246.

Andrew J. Clark and Herbert Scarf. Optimal policies for a multi-echelon inventory problem. *Management Science*, 6(4):475–490, 1960.

Joren Gijsbrechts, Robert N. Boute, Jan A. Van Mieghem, and Dennis J. Zhang. Can deep reinforcement learning improve inventory management? Performance on dual sourcing, lost sales, and multi-echelon problems. *Manufacturing & Service Operations Management*, 24(3):1349–1368, 2022.

Xiao Han, Weijian Zhang, Jiaxin Wang, Fan Zhang, and Jieping Ye. A better match for drivers and riders: Reinforcement learning at Lyft. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 2927–2936, 2022.

Julian Ibarz, Jie Tan, Chelsea Finn, Mrinal Kalakrishnan, Peter Pastor, and Sergey Levine. How to train your robot with deep reinforcement learning: Lessons we have learned. *The International Journal of Robotics Research*, 40(4-5):698–721, 2021.

Dmitry Kalashnikov, Alex Irpan, Peter Pastor, Julian Ibarz, Alexander Herzog, Eric Jang, Deirdre Quillen, Ethan Holly, Mrinal Kalakrishnan, Vincent Vanhoucke, and Sergey Levine. Scalable deep reinforcement learning for vision-based robotic manipulation. In *Conference on Robot Learning*, 2018.

Minne Li, Zhiwei Qin, Yan Jiao, Yaodong Yang, Zheng Gong, Jun Wang, Changjie Wang, Gauge Wu, and Jieping Ye. Efficient ridesharing order dispatching with mean field multi-agent reinforcement learning. In *Proceedings of the 2019 World Wide Web Conference (WWW)*, pages 983–994, 2019.

Jiaxi Liu, Xiaoqing Wang, Yuming Deng, Xingyu Wu, and Yidong Zhang. Dynamic pricing on E-commerce platform with deep reinforcement learning: A field experiment. *arXiv preprint arXiv:1912.02572*, 2019.

Yuriy Nevmyvaka, Yi Feng, and Michael Kearns. Reinforcement learning for optimized trade execution. In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pages 673–680, 2006.

Zhiwei Qin, Xiaocheng Tang, Yan Jiao, Fan Zhang, Zhe Xu, Hongtu Zhu, and Jieping Ye. Ride-hailing order dispatching at DiDi via reinforcement learning. *INFORMS Journal on Applied Analytics*, 51(3):272–286, 2021.

John Rust. Optimal replacement of gmc bus engines: An empirical model of harold zurcher. *Econometrica*, 55(5):999–1033, 1987.

Tom Silver, Kelsey Allen, Josh Tenenbaum, and Leslie Kaelbling. Residual policy learning. *arXiv preprint arXiv:1812.06298*, 2018.

Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, 2nd edition, 2018.

Xiaocheng Tang, Zhiwei Qin, Fan Zhang, Zhaodong Wang, Zhe Xu, Yintai Ma, Hongtu Zhu, and Jieping Ye. A deep value-network based approach for multi-driver order dispatching. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 1780–1790, 2019.

Di Wu, Xiujun Chen, Xun Yang, Hao Wang, Qing Tan, Xiaoxun Zhang, Jian Xu, and Kun Gai. Budget constrained bidding by model-free reinforcement learning in display advertising. In *Proceedings of the 27th ACM International Conference on Information and Knowledge Management (CIKM)*, pages 1443–1451, 2018.